

Decisions

The choices behind SecureGate v0.1: what we chose, why, and what we rejected. Add an entry whenever you make a choice that someone might question later. Words in bold are explained in the Glossary (glossary.pdf).

1. Gitleaks finds, SecureGate decides

- **Chose:** **Gitleaks** finds candidate secrets; our own `policy.yaml` decides what happens to them.
- **Why:** Gitleaks already knows many key formats and is widely used. What we add is the decision and the safe reporting.
- **Rejected:** writing our own detection rules for every provider; TruffleHog in v0.1 (it joined later as a second finder, see 45).

2. One readable policy, first match wins

- **Chose:** rules checked from top to bottom; the first rule that matches decides.
- **Why:** anyone can read the file and predict the result.
- **Rejected:** scores and weights (hard to predict); several policy files.

3. The last rule must match everything

- **Chose:** a policy without a final catch-all rule, or with unknown words in it, is refused (exit code 2).
- **Why:** every finding gets a decision, and a typo can never switch a rule off silently.
- **Rejected:** a hidden default decision.

4. Rule order as specified

- **Chose:** the order of SecureGate's policy table: 1 a key confirmed live, 3 placeholders, 7 tests, fixtures, docs, Markdown and example files, 8 live provider keys, 9 test-mode keys, 10 passwords in the code, 13 risky handling, 14 everything else. (Until Layer 2 it was placeholders, tests/fixtures/docs, provider keys, GitHub tokens, everything else.)
- **Consequence:** a real AWS key inside `docs/`, `tests/` or a `.md` file only warns, unless TruffleHog confirms that it is live. Move rule 7 below rule 8 if that is not wanted.

5. We mask values ourselves, in memory

- **Chose:** Gitleaks reports raw values into a private temporary folder; we read them into memory, delete the folder, and **mask** them before anything is shown.
- **Why:** the policy must see the raw value to recognize placeholders such as `changeme`.
- **Rejected:** Gitleaks' own redaction (it would hide values from the policy too); keeping reports on disk. A possible improvement: read the report from Gitleaks' normal output, so it never touches the disk.

6. Masking shape

- **Chose:** values of 16+ characters show their first 4 and last 4 characters around `****`; shorter values become `****`.

- **Why:** enough to recognize a key, never enough to use it. The number of stars never changes, so the length stays hidden.

7. Fingerprints use a private key

- **Chose:** HMAC-SHA256, keyed by `SECUREGATE_HMAC_KEY` (at least 32 characters) or a random key stored in `.securegate/local.key`.
- **Why:** a plain hash of a short password can be cracked by hashing guesses. Without the key, a **fingerprint** reveals nothing.
- **Consequence:** the finding id is the start of the fingerprint, so the same secret in two places has the same id. The id names the secret, not the place.

8. Exit codes 0 / 1 / 2, and Gitleaks runs with `--exit-code 99`

- **Why:** Gitleaks normally uses 1 both for "found leaks" and for "crashed". A different number for leaks lets us tell them apart.

9. Fail closed

- **Chose:** any problem ends with exit code 2, never 0.
- **Found during the build:** asked to scan a folder that is not a Git repository, Gitleaks logs an error but exits 0 with an empty report. SecureGate now treats any error output from Gitleaks as a failure.

10. Nothing can silence a finding outside the policy

- **Chose:** `gitleaks:allow` comments in code are ignored, and a `.gitleaksignore` file in the scanned folder stops the scan with exit code 2.
- **Why:** otherwise a developer could hide a leak without the policy ever seeing it.
- **Rejected:** a warning only.

11. Gitleaks is only run from the PATH folders

- **Why:** on Windows, a program in the current folder would run first, and a scanned repository could contain a fake `gitleaks.exe`.

12. The ACME Pay rule has no randomness filter

- **Why:** placeholders like `acme_live_xxxx...` should reach the policy, which decides to ignore them. One place decides.

13. Only third-party folders may be skipped

- **Chose:** `.gitleaks.toml` skips Python virtual environments (installed packages). Our own code is never skipped.
- **Why:** if the scanner flags our code, we fix the code. A test scans our own files on every run.

14. No key-shaped text in our files

- **Chose:** fake secrets are built at runtime from a seed and short pieces; markers like PEM headers too.
- **Rejected:** sample keys stored in test files.

15. The demo repo lives outside SecureGate

- **Chose:** `../securegate-demo`, and never inside any Git repository. `--force` only empties a folder the generator made before; it leaves a `.securegate-demo` marker file there.
- **Rejected:** letting `--force` empty any folder.

16. Same seed, same demo repo

- **Chose:** a seed for every random choice, a fixed author (Riya Demo) and dates, and Git run without your personal Git settings.
- **Why:** results can be compared over time; the same seed gives the same commit IDs.

17. The answer sheet holds no values

- **Chose:** `ground_truth.csv` lists file, line, commit, kind and expected decision, never the value itself.

18. The scorecard is reported, not enforced

- **Why:** tests must not push the rules into looking good. First results (seed 42, repo mode): 15 hits, 2 **misses** (a database password inside a web address, a password with symbols), 1 wrong decision (the AWS secret key only warns: Gitleaks catches it with a generic rule), 2 **false alarms** (a client ID and a commit hash used as a cache key).

19. A simple confidence score, for now

- **Chose:** 0.9 for known key formats; for generic matches, $\text{entropy} \div 6$, at most 0.8.
- **Why:** a placeholder until a learned model replaces it. Confidence never decides anything; only the policy does.

20. findings.json has a summary at the top

- **Chose:** status, exit code, Gitleaks version and counts, then the findings. After an error there is no findings list at all.
- **Why:** a failed scan can never be mistaken for a clean one.

21. Tooling

- **Chose:** Python 3.12 (installed next to 3.14), GNU make from winget, setuptools, pytest and ruff; LF line endings in every file.
- **Why:** these match the plan; LF avoids Windows line-ending noise in Git.

22. Small helper modules added during the build

- `demo/app.py`: the fake app's files and commit story. `programs.py`: finds `git` and `gitLeaks` safely. `validate.py`: shared checks for `policy.yaml` and `catalog.yaml`.
- **Why**: one job per module, and no copied code.

23. The dashboard is local and read-only

- **Chose**: a small Flask app that only reads one `findings.json`, answers only on 127.0.0.1 (this computer), and keeps debug off unless `--debug` is given.
- **Why**: findings are sensitive even when masked, so nobody else on the network should reach them, and a report viewer has no reason to change anything.
- **Rejected**: a shared web server, logins and editing (out of scope for now).

24. No JavaScript, and it works offline

- **Chose**: plain HTML and CSS. The Content-Security-Policy header forbids scripts, and every file comes from SecureGate itself.
- **Why**: nothing to download, nothing to break, and nothing for a malicious report to run. It works on a laptop with no internet.
- **Rejected**: JavaScript frameworks and chart libraries.

25. Severity bars are HTML meters

- **Chose**: one `<meter>` element per severity.
- **Why**: screen readers read them as values, and they need no inline styles, which the strict security policy would block.

26. The dashboard checks the masking again

- **Chose**: every `masked_value` must have the masked shape (`****`, or 4 characters + `****` + 4). A report that breaks this is refused, not shown.
- **Why**: a report edited by hand, or written by a future bug, must never put a whole secret on screen.

27. No report means a helpful page

- **Chose**: a missing, broken or failed report shows a page (HTTP 503) with the exact command that creates one. The report is read again for every page.
- **Why**: the first thing a new user sees tells them what to do next, and a new scan appears after a reload.

28. One detail page per secret

- **Chose**: `/findings/<id>` lists every place where that secret was found.
- **Why**: the id names the secret, not the place (see entry 7), and the fix (revoke and replace) is the same for all its places.

29. Design tokens with Fluent 2 names

- **Chose:** every colour, font size, spacing value and radius is a CSS variable in `tokens.css`, named like Fluent 2 tokens. Tests fail if `app.css` uses a raw value, or if text drops below WCAG AA contrast. Screens 1600 pixels and wider (projectors) get larger text and a wider page.
- **Why:** the designers can drop in their Figma tokens by changing values only, without breaking accessibility.

30. The sample report doubles the demo

- **Chose:** `make sample-report` scans a demo repo holding the DemoPay app twice (once under `billing/`), which gives about 20 real findings.
- **Why:** the usual demo scan has only 11 findings, and the designers need real data rather than invented rows.

31. Two gates, and only one of them is the control

- **Chose:** a laptop gate (a pre-commit hook on staged changes) and a merge gate (the `secret-gate` check on every pull request). Only the merge gate is required.
- **Why:** the laptop gate gives the fastest feedback but runs only where it is installed and can be skipped with `--no-verify`. The merge gate runs for every pull request on GitHub.
- **Rejected:** relying on the laptop gate alone.

32. The laptop gate brings its own Python environment

- **Chose:** a pre-commit hook with `language: python` and a small launcher, `tools/precommit_hook.py`, that runs this checkout's SecureGate.
- **Why:** a plain `securegate` command would only work with `.venv` activated, and on Windows a bare `python` is a different Python without SecureGate. After `make hooks` the hook works offline.

33. The merge gate scans every commit of a pull request

- **Chose:** a range scan from the pull request's base to its head, with no path filters on the trigger.
- **Why:** a secret deleted in a later commit is still in the history. A required check with path filters would never run for some pull requests and leave them stuck.

34. The base branch judges each pull request

- **Chose:** the workflow installs SecureGate, and takes `policy.yaml` and `.gitleaks.toml`, from the pull request's base commit.
- **Why:** otherwise a pull request could loosen the policy or change the scanner and pass its own check. Changes to the gate count once merged.
- **Known limit:** a pull request can still edit the workflow file, because GitHub runs the pull request's copy. `merge-gate.md` recommends requiring an approval.
- **Rejected:** `pip install -e .` of the pull request itself, as first specified.

35. Pinned actions and a checked Gitleaks download

- **Chose:** `actions/checkout` and `actions/setup-python` at their latest major version (v7, looked up on 2026-09-30), pinned to full commit SHAs with the version in a comment. Gitleaks comes from its official release, and both the checksums file (its SHA-256 is pinned in the workflow) and the archive are verified before use.
- **Why:** a tag can be moved to other code; a commit SHA cannot. The pinned checksum also protects against a release file being replaced later.

36. The summary is a SecureGate command

- **Chose:** `securegate summary` writes the Markdown for GitHub's job summary, in a step that always runs and never changes the result.
- **Why:** it reuses the dashboard's report checks, so only masked values can reach GitHub's page, and it is tested like the rest of SecureGate.

37. Demo tokens use our invented ACME format

- **Chose:** `securegate demo-token` prints a new random `acme_live_` token each time, and the demo pull request uses only such tokens.
- **Why:** the repository is public. ACME Pay does not exist, so the token unlocks nothing, and GitHub's push protection does not know the format, so the token reaches the pull request and the gate can be shown for real.
- **Rejected:** fake tokens in real formats (Stripe, AWS, GitHub): push protection may stop them, and they look too much like real leaks.

38. The docs are Markdown, with PDF copies

- **Chose:** the pages in `docs/` stay Markdown, and `make docs-pdf` prints each one to `docs/pdf/` with headless Edge, styled with the dashboard's design tokens. A manifest records which version of each page its PDF came from; only changed pages are rebuilt, and a test fails when a PDF is out of date.
- **Why:** GitHub shows Markdown, pull requests can review it, and the gates scan it for secrets. Gitleaks skips PDF files, so PDF-only docs would never be checked.
- **Links between pages** appear in the PDFs as text, such as "Testing (testing.pdf)": the browser would otherwise write this computer's file paths into them.
- **The one diagram** is drawn by Mermaid 12.0.0, loaded from jsDelivr and checked against a pinned hash, so only that step needs the internet.
- **Rejected:** replacing the Markdown with PDFs; reportlab, which would mean laying out every page by hand.

39. The dashboard never shares its port

- **Chose:** SecureGate opens the dashboard's port itself, the way Python's `socket.create_server` does, and hands it to Flask's web server. A busy port stops the dashboard with exit code 2 and says what to do.
- **Why:** Flask's web server (Werkzeug) opens ports with a setting, `SO_REUSEADDR`, that on Windows lets a second server join a port another one is already using. Two dashboards then shared port 5000 without

an error, and which one answered was left to chance. Where the port was really refused, Werkzeug ended the program with exit code 1, SecureGate's code for "blocked".

- **Rejected:** picking a free port automatically: the address would change from run to run, while the docs and the menu promise `127.0.0.1:5000`.

40. A double-click launcher for Windows

- **Chose:** `Start SecureGate.cmd` runs `.venv`'s Python directly, asks before setting anything up, checks the setup with `securegate version`, then opens the menu (entry 41).
- **Why:** `make` should not be needed just to see SecureGate work. A broken setup gets its own hint: delete `.venv`, then double-click again.
- **Rejected:** a PowerShell script: double-clicking one opens it in Notepad.

41. The double-click opens a menu

- **Chose:** `securegate menu` (also `make menu`): a numbered menu in the terminal, under a padlock and the name in big letters. Each choice runs the `securegate` commands a person would type, and shows each command before running it. Choice 1 builds the demo only when it is missing, scans it every time, and offers the dashboard only after a scan that ended with exit code 0 or 1. Enter closes the dashboard and brings the menu back.
- **Why:** SecureGate should open like an app for people who don't type commands, yet stay exactly the tool the gates use: the same commands, exit codes and masking. A rebuild would undo changes made to the demo for a presentation; a fresh scan follows the current `policy.yaml`; after a failed scan, the dashboard would only show the failure. Enter rather than Ctrl+C, because after Ctrl+C, `cmd.exe` asks "Terminate batch job (Y/N)?".
- **Colour** only in a real terminal, and never when `NO_COLOR` is set. ASCII letters when the terminal cannot show block characters. Q ends the menu with exit code 0; if its input ends, it stops with exit code 2, so it can never stand in for a gate.
- **Rejected:** a terminal-UI library such as Rich or Textual (a new dependency); answering with a single key press (Windows-only keyboard calls, and tests could not type the answers); the dashboard in a second window (one more window to close).

42. A dark dashboard, with downloads and a printable report

- **Chose:** a dark theme, a cool slate with one azure accent, and light colours when printed. The overview leads with the result in words (BLOCKED or PASS), then four tiles, the blocked findings with the most severe first, and the severity bars. The findings download as CSV, JSON or the Markdown summary, and `/report` puts the whole report on one page, to print or to save as a PDF.
- **Why:** dark reads well on a laptop and on a projector in a dim room, and light paper is cheaper to print and easier to read. Downloads are made from the same checked report the pages show, never by handing out `findings.json` as it is, so they can only hold masked values and the fields SecureGate knows. In the CSV, a cell that starts with `=`, `+`, `-` or `@` gets an apostrophe in front, because spreadsheet programs run such cells as formulas (the usual advice against CSV injection); a masked private key starts with dashes. The JSON download is a SecureGate report, so the dashboard and `securegate summary` can open it.
- **Severity dots** use status colours (critical red, high orange, medium amber, low and info grey), always next to the severity's name. The bars keep the one accent colour.

- **Rejected:** making PDFs on the server (a new dependency such as WeasyPrint, while the browser's Save as PDF already does it); buttons that print or download with JavaScript (the dashboard forbids scripts); a light/dark switch (it needs JavaScript, or a setting to remember).

43. Rules keep their number from the policy table

- **Chose:** each rule in `policy.yaml` carries its `number` from SecureGate's 14-rule policy table, and every report names the rule that decided as "rule 8: provider-keys" (the `matched_rule` field). Numbers go up from top to bottom; gaps are allowed, because rules 2, 4, 5, 6, 11 and 12 come later. Error messages use the same numbers, such as `rule 10 ('hardcoded-passwords')`.
- **Why:** the table is how the team talks about the policy, so a report and a conversation should use the same name. The `reason` still starts with the rule's name, so older readers keep working.
- **Rejected:** numbering rules by their place in the file (adding one rule would renumber all the others).

44. The live-key rule looks at the key, not at the scanner's rule

- **Chose:** rule 8 matches the format of the value: `acme_live_`, `sk_live_` and `rk_live_`, AWS key ids (AKIA and its relatives), Bedrock keys, GitHub tokens (`ghp_`, `gho_`, `ghu_`, `ghs_`, `ghr_`, `github_pat_`) and private keys. Test-mode keys (`acme_test_`, `sk_test_`) only warn (rule 9).
- **Why:** four scanners name their rules differently, but a live key looks the same to all of them. Gitleaks' Stripe rule also matches test keys, which cannot move real money. Everything that Layer 1 blocked by rule name is still blocked.
- **Consequence:** a live key blocks as "high"; only a key that TruffleHog confirms is live is "critical" (rule 1).
- **Rejected:** matching scanner rule names (they differ per scanner and mix live and test keys).

45. TruffleHog is the second required scanner

- **Chose:** `--scanners all` adds TruffleHog to Gitleaks. Like Gitleaks it is required: if it is asked for and is missing, crashes or writes something unreadable, the scan ends with exit code 2. It runs with `--no-ignore-tag` (a `trufflehog:ignore` comment cannot hide a finding), `--fail-on-scan-errors` (a partial scan is an error) and `--no-update` (it never replaces itself with a version nobody pinned). Only its `Raw` field is read, and only to mask it; the other fields that can hold the secret, and the text of its logs, are never kept. It only scans Git history, so it is skipped in `dir` and `staged` modes, and the default stays Gitleaks alone (the laptop gate stays fast).
- **Why:** TruffleHog can ask the provider whether a key still works, which turns "looks like a key" into "is a live key" (rule 1).
- **Tests** run it only with `--no-verification`, so a fake key is never sent to a real provider. `.trufflehog.yaml` adds a detector for ACME Pay tokens without a check address, so TruffleHog contacts nobody about them.
- **Rejected:** `--fail` (its exit code 183 hides crashes among findings); reading TruffleHog's own `Redacted` text (it can show most of the key).

46. One finding per secret and line

- **Chose:** each scanner's finding is decided by the policy on its own; then the findings of the same key in the same file and commit become one, even if TruffleHog's line number is slightly off. A finding of

Semgrep or Bandit is added to the secret found on its file and line, instead of becoming a finding of its own. The merged finding keeps the strongest decision (with that rule's reason and fix), the highest severity, the strongest live check, and every scanner that found it (`detectors`).

- **Why:** one leak should be one line in the report, with every piece of evidence about it, not four lines from four tools.
- **Rejected:** deciding once on a merged finding (a risky-handling warning on the same line as a placeholder would be lost).

47. Pinned scanners, installed and checked by `make scanners`

- **Chose:** the versions of TruffleHog, Semgrep and Bandit live in one place, the `env:` block at the top of the merge gate's workflow, and `CLAUDE.md` repeats them (a test checks that they match). `make scanners` installs Semgrep and Bandit into `.venv` with pip, and downloads TruffleHog from its GitHub release: the release's list of checksums must match a pinned SHA-256, the archive must match its line in that list, and only the program is taken out of the archive, into `.venv`'s scripts folder. SecureGate looks for programs on PATH and in that folder, never in the current folder.
- **Why:** the laptop and GitHub run the same versions, and a download is checked before it runs.
- **Rejected:** `curl | sh` install scripts (nothing is checked before it runs); winget (it has no TruffleHog package).

48. Semgrep and Bandit read a private copy of the code, and are optional

- **Chose:** `--scanners all` also runs Semgrep (Semgrep's p/secrets rules plus our own `rules/securegate-risky.yml`) and Bandit (its password checks B105, B106 and B107). They read code, not history: a private copy of the files the range changed, as they are at its end (in a repo scan, every tracked file at HEAD), taken from Git with `git archive`. Ignore files (`.semgrepignore`, `.gitignore`, `.bandit`) are left out of the copy and an empty `.semgrepignore` is written instead; Semgrep runs with `--disable-nosem` and Bandit with `--ignore-nosec`, so neither a file nor a comment in a pull request can hide code from them. If one of them fails or is missing, the scan goes on and the report records that it did not run. If p/secrets cannot be downloaded, Semgrep runs our own rules alone and the report says so.
- **Why:** they add evidence that Gitleaks and TruffleHog cannot give: passwords next to their names, and code that leaks a secret into a log or a URL. But they look at code, not at secrets in history, so a pull request should not be stopped just because one of them could not run.
- **Values:** only the rule, the file and the line are read from their output. Semgrep's `extra.lines` and `extra.message`, and Bandit's `code` and `issue_text`, quote the secret and are never kept: the value is cut out of our own copy (Semgrep) or out of Bandit's message, in memory, and masked at once. Our own Semgrep rules point at code, not at a secret, so their findings show `****` and nothing of the line.
- **Rejected:** scanning the working tree (it may not be the commit being judged); letting Semgrep use the repository's ignore files; `python -m semgrep` (deprecated by Semgrep).

49. One Markdown report for the pull request, and SARIF from our own findings

- **Chose:** `securegate scan --comment FILE --summary FILE --sarif FILE`. The comment and the job summary are the same report from one template: a one-line verdict (BLOCKED, PASSED WITH

WARNINGS, PASSED, or ERROR when the scan did not finish), a table (decision, rule, file and line, masked value, the scanners that found it, the live check), a rotation checklist for every blocked key (ACME Pay, Stripe, AWS, GitHub, private keys, or a general one) ending with "Deleting the line is not enough: the key stays in Git history.", why each warning was not blocked, and the ignored findings folded away. The comment starts with a hidden marker, so the workflow can update its own comment instead of adding one per run. A scanner that did not run shows as a red CAUTION note. The SARIF file has one rule per policy rule, with its severity, and only the blocks and warnings.

- **Why:** everything a reviewer needs is in the pull request itself, in plain words. Building every output from `findings.json`, read back through the dashboard's check that every value is masked, means no output can hold more than that check let through, and every value is escaped so a file name cannot break the table or add HTML.
- **Consequences:** after an error, the comment and summary say ERROR and any old SARIF file is deleted, because uploading an empty one would tell GitHub that every earlier alert was fixed. Long reports stop after 50 rows (GitHub refuses comments over 65,536 characters); `findings.json` keeps everything.
- **Rejected:** uploading the scanners' own SARIF (it holds unmasked values and none of our decisions); a red text colour through GitHub's maths notation (it does not degrade gracefully where it is not rendered).